Advanced Topics in Computer Graphics I

Universität Bonn
Institut für Informatik II
April 30, 2026

Summer term 2026
Prof. Dr. Reinhard Klein

Domenic Zingsheim

Sheet R03 - Radiosity

Hand in your solutions via eCampus by Fr, 08.05.2026, **08:00 a.m.**. Compile your solution to the theoretical part into a single printable PDF file. For the practical part, hand in a single ZIP file containing only the exercise* folder within the src/ directory. Please refrain from sending the entire framework.

Assignment 1) Radiosity *(7 Pts)*

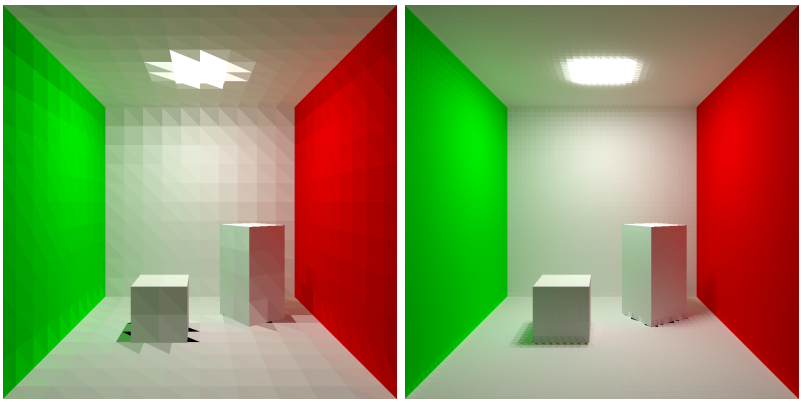

Figure 1: Cornell Box scene rendered with the Radiosity method. To obtain these renderings, run `./bin/exercise03_Radiosity -s data/exercise03_Radiosity/cornell_box.xml` and `./bin/exercise03_Radiosity -s data/exercise03_Radiosity/cornell_box_fine.xml` from the project root, respectively.

In this exercise, the Radiosity method has to be implemented. It is integrated into our framework using a `RadiosityEmitter` class and a `RadiosityRayGenerator`. The `RadiosityEmitter` is simply a surface emitter that wraps the surface parameters emission and albedo, as well as the per-triangle radiosity solution. The `RadiosityRayGenerator` is responsible for all computation in this case. It first computes the fundamental matrix, then solves a linear equation system for the radiosity and also generates images for a camera. We interpret each triangle primitive as a finite element in the radiosity method, so the radiosity is computed per triangle and not per vertex in a mesh.

a) Complete the `__raygen__generateRadiosity` program in `radiosityraygenerator.cu` by computing the entries F_{ij} of the form factor matrix F, taking the occlusion between two surface patches into account as described in the lecture.

b) Complete the `RadiosityRayGenerator::computeRadiosity()` method in `radiosityraygenerator.cpp` by solving

$$\underbrace{(\mathbf{I} - \mathrm{diag}(\mathbf{r}) \cdot F)}_{=:K} \cdot B = E \tag{1}$$

for each color channel, where $\mathbf{I}$ is the identity matrix, $\mathbf{r}$ is the albedo for each primitive, F is the form factor matrix and E describes the emission of each primitive in the scene. Inverting

the matrix K is usually not feasible. Therefore a typical approach for solving this is the iterative Jacobi method[1]: After initializing $B^{(0)} = E$, iterate

$$B^{(i+1)} = B^{(i)} + \lambda \cdot (E - K \cdot B^{(i)}) \tag{2}$$

until convergence, where λ is a user defined parameter.

Theoretical Assignments

Assignment 2) Reflection Equation *(5 + 1 Bonus Pts)*

Consider the simple scene from the first sheet as it is shown in Figure 2. Again, we're given the radiance of the sun $L_s = 20.045 \frac{MW}{m^2 sr}$ and the direction v having an angle of 45° to the table. You can again assume that the radiance is constant over the whole surface of the sun.

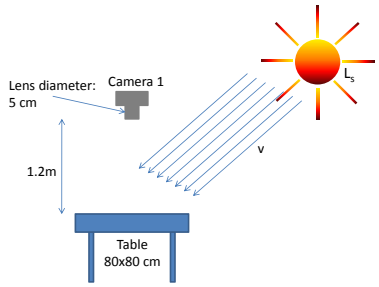

Figure 2: Toy example for radiometric quantities

Now, assume that the table plate is perfectly diffuse with 50% albedo. A camera having a lens of 5cm diameter has been placed 1.2m above the center of the table looking exactly downwards. Calculate the irradiance and the radiant power at the camera lens assuming that everything else around the table is perfectly black. Furthermore, you can for simplicity assume that the table covers the same solid angle from all points of the lens and that the irradiance is constant over the camera lens. Use tools like Maple or WolframAlpha[2] for solving the integrals.

For a rough approximation you will get 5 points and for a complete solution you will get an additional bonus point.

Assignment 3) Neumann Series Rendering Equation *(3Pts)*

The Neumann series representation of the rendering equation is given as

$$L = \sum_{i=0}^{\infty} T^i L_e$$

a) Define the quantities L, L_e, and T.

b) Describe the meaning of the summands $T^0 L_e$, $T^1 L_e$, and $T^2 L_e$.

c) Give an expression for the solution of the rendering equation using the Neumann series.
Hint: We are looking for an expression $L = \ldots$ without infinite sums.

Good luck!

[1] https://en.wikipedia.org/wiki/Jacobi_method
[2] https://www.wolframalpha.com